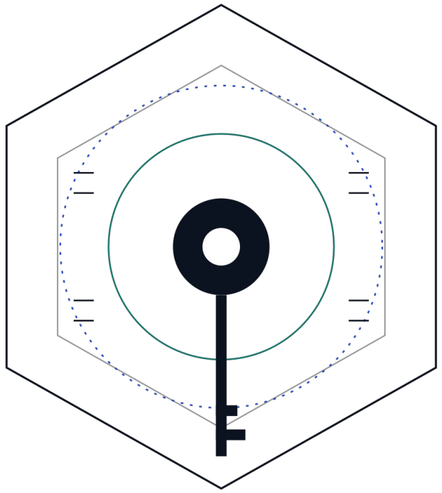
Buổi 06

Authentication *và* Authorization

Xác thực và phân quyền trong API: HTTP Basic, JWT, OAuth 2.0 và các quyết định triển khai.



Nội dung

Nền tảng & phân biệt khái niệm

Vì sao API cần auth · AuthN ≠ AuthZ

Basic Auth & API Key

RFC 7617 · header Authorization · secret key qua header

JWT — cấu trúc & triển khai

header / payload / signature · Flask-JWT-Extended · refresh token

OAuth 2.0 & OIDC

4 vai trò · chọn grant · Authorization Code + PKCE · OIDC

Phân quyền · RBAC & Scope

so sánh mô hình · decorator @require_role trong Flask

Lỗ hổng

Argon2id/bcrypt · bảo vệ credential · OWASP API Security

Mục tiêu

- Phân biệt vai trò của OAuth 2.0 và JWT trong API REST
- Giải thích và triển khai ví dụ JWT, refresh token và kiểm tra quyền bằng Flask

Hai câu hỏi đầu tiên mọi API public phải trả lời được

API cần xác định caller khi cần và kiểm tra quyền đối với từng hành động, từng tài nguyên.

CASE 1 · 2018

Facebook · Cambridge Analytica

Graph API v1 cho phép ứng dụng thu thập dữ liệu của người dùng và bạn bè chưa tương tác với ứng dụng (FTC, 2019).

VẤN ĐỀ

Quyền truy cập dữ liệu bạn bè và thông báo thu thập dữ liệu không đầy đủ.

Bài học: giới hạn dữ liệu và giải thích rõ mục đích sử dụng.

CASE 2 · 2022

Optus

Sự cố được công bố tháng 9/2022. OAIC cáo buộc thiếu biện pháp bảo vệ dữ liệu của khoảng 9,5 triệu người (2025).

VẤN ĐỀ

Nguồn OAIC xác nhận quy mô và cáo buộc bảo vệ dữ liệu; không xác nhận endpoint minh họa cụ thể.

Bài học: kiểm tra bề mặt API và quyền truy cập dữ liệu.

CASE 3 · GIẢ ĐỊNH

Secret bị commit lên Git

Ví dụ giả định: API key còn hiệu lực trong repository công khai có thể bị dùng để gọi API bằng quyền của chủ key.

VẤN ĐỀ

Key có quyền rộng · phát hiện lộ chậm · thu hồi chưa đầy đủ

Bài học: quyền tối thiểu, phát hiện secret và thu hồi khi lộ.

NGUYÊN TẮC: công khai endpoint theo chủ đích; bảo vệ endpoint riêng tư bằng xác thực và chính sách truy cập.

Các cơ chế thường gặp trong bảo vệ API

Các cột thuộc những lớp khác nhau: cách mang credential, cách quản lý phiên, định dạng token và framework ủy quyền.

So sánh	① ĐƠN GIẢN HTTP Basic JPG	② MÁY-MÁY API Key JPG	③ WEB CỔ ĐIỂN Cookie-Session JPG	④ STATELESS JWT JPG	⑤ DELEGATION OAuth 2.0 JPG
Định danh bằng	user:pass base64 mỗi request, đặt trong header Authorization	Secret ngẫu nhiên đủ entropy, gửi theo giao diện API quy định	Cookie session ID (chuỗi random, opaque) trình duyệt tự gửi lại	Token tự chứa (claims), server không cần nhớ session	Access token do Authorization Server cấp theo một grant
State ở server?	Stateless; thường lookup user trong DB mỗi request	Thường tra cứu hash và metadata của key ở server	**Stateful** server giữ session store (Redis, DB)	Stateless (verify chữ ký) revocation là vấn đề	Tùy token: kiểm chữ ký hoặc tra cứu/introspection
Điểm yếu	Credential gửi lại nhiều lần; thu hồi bằng khóa tài khoản hoặc đổi credential.	Key bị lộ có thể bị dùng trong phạm vi quyền được cấp.	Cookie tự gửi có rủi ro CSRF; cần kiểm soát phù hợp.	Cần kiểm chữ ký, claims và thiết kế thu hồi token.	Sai redirect URI, CSRF hoặc rò rỉ token có thể phá flow.
Phù hợp khi	Internal tool ngắn hạn, prototype, hoặc tích hợp HTTPS-only giữa hai service tin tưởng nhau.	Tích hợp máy-máy và API cho developer.	SSR web app cùng domain, BFF serving single-page app trên HTTPS.	Mobile API, microservice nội bộ, scenarios không cần revoke tức thì.	Ứng dụng truy cập tài nguyên được ủy quyền; đăng nhập dùng OIDC.
Dùng cho production?	**TÙY BỐI CẢNH**	**⚠️ CẨN THẬN**	**√ ĐƯỢC (web)**	**√ ĐƯỢC**	**√ ĐƯỢC (chuẩn)**

Những cơ chế này có thể kết hợp. OAuth không quy định token phải là JWT; cookie cũng có thể mang token.

Authentication *vs* Authorization

AuthN xác minh danh tính; AuthZ quyết định quyền. Có thể dùng chung middleware nhưng phải tách rõ trách nhiệm.

AUTHENTICATION

Bạn là *ai*?

Xác minh **danh tính** của subject đang gửi request — trả lời xong thì có một *principal* (thường là user_id).

CÁC YẾU TỐ XÁC THỰC

Biết	Có	Là
password · PIN	phone · smart card · token · key	vân tay · Face ID · mống mắt

TRẢ LỜI ĐƯỢC DẠNG

```
POST /login → 200 OK
// principal: subject = User{ id: 42, email: "an@..." }
```

AUTHORIZATION

Bạn được làm *gì*?

Áp chính sách cho subject và ngữ cảnh để quyết định hành động trên tài nguyên; có thể có subject ẩn danh.

CÁC MÔ HÌNH PHÂN QUYỀN

RBAC	ABAC
user có role → role có permission	attribute (role, time, IP) → policy

TRẢ LỜI ĐƯỢC DẠNG

```
GET /orders/123
// 403 nếu 123.owner != current_user.id
```

VÍ DỤ PHÂN BIỆT — HỆ THỐNG NGÂN HÀNG

- AuthN xác minh khách hàng bằng mật khẩu và OTP.
- AuthZ cho phép xem tài khoản của mình; hạn mức chuyển tiền do chính sách ứng dụng quy định.

HTTP Basic Auth *— đơn giản tới mức đáng ngờ*

RFC 7617 (2015) cập nhật Basic Auth đã có trước đó. Base64 của username:password là encoding, không bảo mật nội dung.

BƯỚC 1 — CHALLENGE

```
# client chưa có credentials
GET /api/orders HTTP/1.1
Host: api.example.com
← response
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="orders", charset="UTF-8"
Content-Type: application/json
```

BƯỚC 2 — CLIENT GỬI LẠI

```
# curl -u (Chrome/Firefox mở popup)
GET /api/orders HTTP/1.1
Host: api.example.com
Authorization: Basic YW5AdmN1LmVkdS52bjpzZWNyZXQxMjM=

# base64 decode → user:pass
```

DEMO NHANH — base64 KHÔNG PHẢI MÃ HOÁ

```
$ python3 -c "import base64;
  print(base64.b64decode('YW5AdmN1LmVkdS52bjpzZWNyZXQxMjM=').decode())"
→ an@vnu.edu.vn:secret123
```

Ai lấy được header Basic có thể giải mã credential. HTTPS bảo vệ khi truyền; log và máy khách vẫn cần được bảo vệ.

Khi nào nên dùng

① Tích hợp đơn giản có HTTPS

② Công cụ nội bộ với credential riêng

③ Demo có kiểm soát

④ CLI với tài khoản quyền hạn chế

Không nên dùng khi

① Trình duyệt cần quản lý phiên linh hoạt

② Truyền HTTP không có TLS

③ Cần delegation và scope theo token

④ Cần logout/thu hồi phiên độc lập mật khẩu

Triển khai Basic Auth trong Flask — xác thực mật khẩu

app.py

```
# requirements.txt: flask, flask-httpauth, bcrypt
from flask import Flask, jsonify, request
from flask_httpauth import HTTPBasicAuth
import bcrypt

app = Flask(__name__)
auth = HTTPBasicAuth()
# KHÔNG BAO GIỜ lưu plaintext — hash khi khởi tạo
_users = {
    "an@vnu.edu.vn": bcrypt.hashpw(
        b"correct horse battery staple", bcrypt.gensalt())
}

@auth.verify_password
def verify_password(username, password):
    if username not in _users:
        return False
    stored = _users[username]
    pwd = (password.encode()
           if isinstance(password, str) else password)
    # Xác minh hash bằng thư viện
    return username if bcrypt.checkpw(pwd, stored) else None
```

Kiểm thử

```
# Demo local HTTPS: -k chỉ cho chứng chỉ tự ký
$ curl -ki https://localhost:5000/api/orders
# 401 nếu thiếu hoặc sai credential

$ curl -ki -u an@vnu.edu.vn:wrong \
  https://localhost:5000/api/orders
# curl sẽ hỏi mật khẩu đúng của ví dụ

$ curl -ki -u an@vnu.edu.vn \
  https://localhost:5000/api/orders
# 200: orders + principal
```

Lưu ý khi review code

① Lưu plaintext hoặc hash nhanh
② Lỗi/thời gian phản hồi làm lộ tài khoản
③ Thiếu rate-limit
④ HTTP hoặc log credential
⑤ Thiếu chính sách quyền trên route

> Mọi cơ chế sau (JWT, OAuth) đều kế thừa những vấn đề này.

Triển khai Basic Auth trong Flask — route và xử lý lỗi

app.py

```
@auth.error_handler
def on_auth_error(status):
    return jsonify({"error": "unauthorized"}), 401, {
        "WWW-Authenticate": 'Basic realm="orders"'
    }

@app.route("/api/orders", methods=["GET"])
@auth.login_required
def list_orders():
    return jsonify({
        "orders": ["o-001"],
        "principal": auth.current_user()
    })

if __name__ == "__main__":
    app.run(ssl_context="adhoc")  # chỉ demo local
```

Kiểm thử

```
# Demo local HTTPS: -k chỉ cho chứng chỉ tự ký
$ curl -ki https://localhost:5000/api/orders
# 401 nếu thiếu hoặc sai credential

$ curl -ki -u an@vnu.edu.vn:wrong \
  https://localhost:5000/api/orders
# curl sẽ hỏi mật khẩu đúng của ví dụ

$ curl -ki -u an@vnu.edu.vn \
  https://localhost:5000/api/orders
# 200: orders + principal
```

Lưu ý khi review code

① Lưu plaintext hoặc hash nhanh
② Lỗi/thời gian phản hồi làm lộ tài khoản
③ Thiếu rate-limit
④ HTTP hoặc log credential
⑤ Thiếu chính sách quyền trên route

> Mọi cơ chế sau (JWT, OAuth) đều kế thừa những vấn đề này.

API Key — credential cho truy cập API

API key có thể gắn với ứng dụng, dự án hoặc tài khoản người dùng; ngữ nghĩa phụ thuộc nhà cung cấp.

MỘT KEY NHÌN NHƯ THẾ NÀY

```
sk_live_aBc3·9dEfG2hI·jKlM4nO5p·qRsT6uV7wX8yZ
```

Ví dụ định dạng tự thiết kế:

- `prefix_randomid_checksum`
- `randomid: sinh từ CSPRNG`
- `checksum: phát hiện lỗi gõ, không chống giả mạo`

BA CÁCH TRUYỀN KEY — CÁCH NÀO PHỔ BIẾN HƠN

```
① Authorization: Bearer <secret>
② X-Api-Key: <secret>
③ ?api_key=<secret>
```

Chọn theo đặc tả API, tránh URL cho secret.

LÝ DO KHÔNG TRUYỀN QUA QUERY STRING

Secret trong URL dễ xuất hiện trong log và lịch sử trình duyệt. Mức rò qua Referer phụ thuộc chính sách và ngữ cảnh.

Real-world patterns

Stripe — RESTful payments

Hai loại: **pk_live_...** (publishable, frontend, → browser OK) và **sk_live_...** (secret, server-only, → lộ là revoke và rotate).

GitHub — OAuth Apps & PAT

PAT gắn với tài khoản người dùng; quyền và thời hạn tùy loại token và chính sách.

AWS — Sig v4 (không phải key đơn)

SigV4 ký request bằng credential AWS. Timestamp giới hạn cửa sổ replay, không loại bỏ mọi replay.

OpenAI — LLM API

Bearer secret là một mẫu triển khai; quyền và hạn mức phụ thuộc cấu hình dự án/API.

> Không có định dạng hoặc chu kỳ 90 ngày chung cho mọi key. Theo chính sách nhà cung cấp và rủi ro của ứng dụng.

Triển khai API Key trong Flask — tạo key và lưu hash

Code minh họa từng phần; helper DB, policy và middleware cần được tích hợp riêng.

keys.py

```python
# 1. Tạo key 256-bit
import secrets, hashlib

def mint_key(prefix):
    raw = secrets.token_urlsafe(32)  # CSPRNG
    body = raw
    digest = hashlib.sha256(raw.encode()).hexdigest()[:8]
    plain = f"{prefix}_{body}_{digest}"  # hiện 1 lần
    store = hashlib.sha256(plain.encode()).hexdigest()
    return plain, store  # lưu hash vào DB

# Hash toàn bộ key được gửi, không cắt chuỗi random.
# Prefix có thể chứa dấu "_" (ví dụ: sk_live).
# Dùng @require_api_key(scopes=[scope]) trên handler.
```

SCHEMA api_keys

```
id          INT PK
prefix      VARCHAR(16)
            # secret prefix
hash        VARCHAR(64)
            # sha256(key), không plaintext
last4       CHAR(4)  # hiển thị UI
owner_id    INT FK users.id
scopes      VARCHAR(128)
            # read,write,...
created_at  TIMESTAMP
expires_at  TIMESTAMP NULL  # vô hạn
revoked_at  TIMESTAMP NULL  # vô hiệu
ip_allowlist TEXT  # danh sách CIDR
```

test với curl

```
# Request minh họa, không giao dịch thật
$ curl -i https://example.test/pay \
  -H "X-Api-Key: <secret>"
# 401: thiếu/key không hợp lệ
# 403: thiếu scope
# Thành công: theo đặc tả endpoint

Thu hồi khi lộ. Rotate theo rủi ro.
```

Triển khai API Key trong Flask — xác minh key

Code minh họa từng phần; helper DB, policy và middleware cần được tích hợp riêng.

keys.py

```
# 2. Decorator verify – lấy key và tra cứu
from functools import wraps
from flask import request, jsonify, g

def require_api_key(scopes=None):
 def d(fn):
  @wraps(fn)
  def w(*a, **kw):
key = request.headers.get("X-Api-Key", "")
if not key:
 return jsonify({"e": "missing"}), 401
hashed = hashlib.sha256(key.encode()).hexdigest()
row = db.fetchone(
 "SELECT id, owner_id, scopes, revoked_at, expires_at "
 "FROM api_keys WHERE hash=?", (hashed,))
if (not row or row["revoked_at"] or
 (row["expires_at"] is not None and
  row["expires_at"] <= now_utc())):
 return jsonify({"e": "invalid"}), 401
# Tiếp: kiểm tra scope và gọi handler.
```

SCHEMA api_keys

```
id          INT PK
prefix      VARCHAR(16)
            # secret prefix
hash        VARCHAR(64)
            # sha256(key), không plaintext
last4       CHAR(4)  # hiển thị UI
owner_id    INT FK users.id
scopes      VARCHAR(128)
            # read,write,...
created_at  TIMESTAMP
expires_at  TIMESTAMP NULL  # vô hạn
revoked_at  TIMESTAMP NULL  # vô hiệu
ip_allowlist TEXT  # danh sách CIDR
```

test với curl

```
# Request minh họa, không giao dịch thật
$ curl -i https://example.test/pay \
  -H "X-Api-Key: <secret>"
# 401: thiếu/key không hợp lệ
# 403: thiếu scope
# Thành công: theo đặc tả endpoint

Thu hồi khi lộ. Rotate theo rủi ro.
```

Triển khai API Key trong Flask — scope và owner

Code minh họa từng phần; helper DB, policy và middleware cần được tích hợp riêng.

keys.py

```
# 3. Tiếp thân hàm w — scope, owner và handler
            granted = set(row["scopes"].split(","))
            if scopes and not set(scopes).issubset(granted):
                return jsonify({"e": "scope"}), 403
            g.api_key_id = row["id"]
            g.owner_id = row["owner_id"]
            g.api_scopes = granted
            return fn(*a, **kw)
        return w
    return d

# Dùng: @require_api_key(scopes=[scope]) trên handler.
# Hàm w nối tiếp phần xác thực ở slide trước.
```

SCHEMA api_keys

```
id           INT PK
prefix       VARCHAR(16)
             # secret prefix
hash         VARCHAR(64)
             # sha256(key), không plaintext
last4        CHAR(4)  # hiển thị UI
owner_id     INT FK users.id
scopes       VARCHAR(128)
             # read,write,...
created_at   TIMESTAMP
expires_at   TIMESTAMP NULL  # vô hạn
revoked_at   TIMESTAMP NULL  # vô hiệu
ip_allowlist TEXT  # danh sách CIDR
```

test với curl

```
# Request minh họa, không giao dịch thật
$ curl -i https://example.test/pay \
  -H "X-Api-Key: <secret>"
# 401: thiếu/key không hợp lệ
# 403: thiếu scope
# Thành công: theo đặc tả endpoint

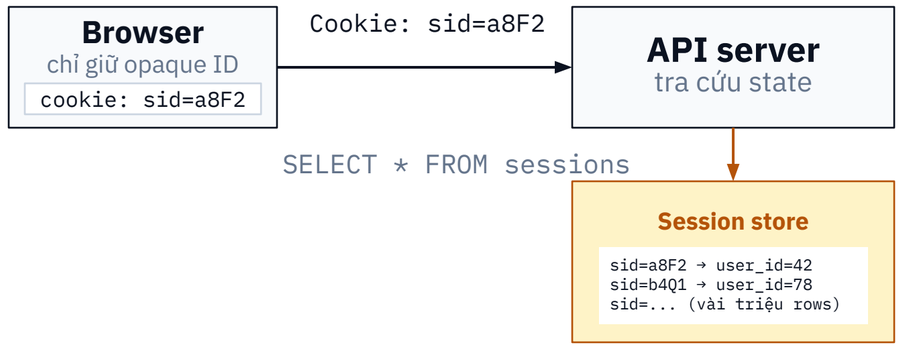
Thu hồi khi lộ. Rotate theo rủi ro.
```

Token-based auth *vs* Server session

So sánh phiên tra cứu ở server với JWT tự chứa. Cookie và Bearer là cách mang credential, không quyết định nơi lưu state.

SERVER-SIDE SESSION

State ở server



+ ƯU

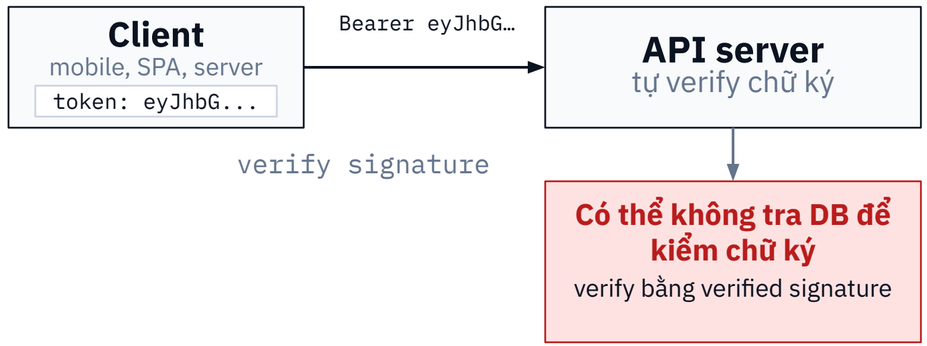
Thu hồi phiên tập trung · cookie chỉ mang ID khó đoán

– NHƯỢC

Cần quản lý session store, vòng đời cookie và CSRF

TOKEN TỰ CHỨA (JWT ĐƯỢC KÝ)

State ở token



+ ƯU

Có thể kiểm chữ ký cục bộ · dùng được ở nhiều loại client

– NHƯỢC

Thu hồi và quyền thay đổi cần thiết kế riêng; Bearer lộ có thể bị dùng lại.

Chọn theo nhu cầu thu hồi, client và hạ tầng. Ví dụ access 15 phút + refresh trong DB chỉ là một cấu hình minh họa.

JSON Web Token — ba phần ghép bằng dấu chấm

Ví dụ JWT dạng JWS compact có ba phần header.payload.signature. JWT dạng JWE được mã hóa có năm phần.

PHẦN 1 · HEADER

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

base64url-safe

PHẦN 2 · PAYLOAD (claims)

```
eyJzdWIiOiI4MjQyMiIsInJvbGUiOiJ2aWV3ZXIiLCJle
HAiOjE3MjQwMTIwMDAsImlhdCI6MTcyNDAwODQwMH0
```

đọc được plaintext — KHÔNG đặt secret ở đây

PHẦN 3 · SIGNATURE

```
HMACSHA256(base64url(header) +
"." + base64url(payload), secret)
```

HEADER · DECODE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- alg: thuật toán bảo vệ token, phải thuộc allow list
- typ: kiểu token, là header tùy chọn

PAYLOAD · DECODE

```
{
  "sub": "82422",
  "role": "viewer",
  "exp": 1724012000,
  "iat": 1724008400
}
```

SIGNATURE · KIỂM CHỨNG BẰNG KHÓA

```
zpXCHhzSkQ2hHY-VdAbygn

7aZBnfj5rX9pJZA9Se6MI
```

- Ví dụ HS256 dùng HMAC bảo vệ toàn vẹn; không che payload.

CLAIMS TIÊU CHUẨN

iss issuer	**sub** subject	**aud** audience	**exp** expiration	**iat** issued at	**nbf** not before	**jti** JWT id (unique)

> Kiểm chữ ký chưa đủ: còn phải kiểm exp, issuer, audience và quy tắc ứng dụng. Thu hồi sớm thường cần state bổ sung.

JWT trong Flask — cấu hình

app.py — Flask-JWT-Extended

```
# Cấu hình
from flask import Flask, request, jsonify
from flask_jwt_extended import (
    JWTManager, create_access_token, create_refresh_token,
    jwt_required, get_jwt_identity, get_jwt
)
import bcrypt, os

app = Flask(__name__)
app.config.update(
    SECRET_KEY=os.environ["FLASK_SECRET_KEY"],
    JWT_SECRET_KEY=os.environ["JWT_SECRET_KEY"],
    JWT_ACCESS_TOKEN_EXPIRES=900,
    JWT_REFRESH_TOKEN_EXPIRES=2592000,
    JWT_ALGORITHM="HS256"
)
jwt = JWTManager(app)
# Secret sinh ngẫu nhiên, cấp qua secret manager/env.
```

FLOW NGƯỜI DÙNG

Vòng đời một phiên

```
1 · Login
POST /api/auth/login
→ {access, refresh}
2 · Gọi API
Authorization: Bearer <access>
3 · Access hết hạn
→ cần refresh hoặc đăng nhập lại
4 · Refresh
Kiểm phiên, cấp access mới
5 · Logout cần bổ sung
Thu hồi access và refresh/phiên
Tham số 15 phút/30 ngày: minh họa
```

REVOCATION — LƯU jti VÀO REDIS

Blocklist thu hồi jti được ghi. Logout còn phải vô hiệu refresh token/phiên liên quan; ví dụ chưa cài route logout.

```
@jwt.token_in_blocklist_loader
def check_revoked(_h, claims):
    jti = claims["jti"]
    return redis.get(
        f"revoked:{jti}"
    ) is not None
```

JWT trong Flask — login và claims

app.py — Flask-JWT-Extended

```python
# Login — cấp access + refresh kèm claims
@app.post("/api/auth/login")
def login():
    user = db.fetchone(
        "SELECT id, pwd_hash FROM users WHERE email=?",
        (request.json.get("email"),))
    pwd = request.json.get("password", "").encode()
    if not user or not bcrypt.checkpw(pwd, user["pwd_hash"]):
        return jsonify(error="invalid_credentials"), 401
    # Helper trả {"roles": [...], "scopes": [...]}.
    claims = load_permissions(user["id"])
    uid = str(user["id"])
    return jsonify(
        access=create_access_token(
            identity=uid, additional_claims=claims),
        refresh=create_refresh_token(
            identity=uid, additional_claims=claims)
    )
```

FLOW NGƯỜI DÙNG

Vòng đời một phiên

```
1 · Login
POST /api/auth/login
→ {access, refresh}
2 · Gọi API
Authorization: Bearer <access>
3 · Access hết hạn
→ cần refresh hoặc đăng nhập lại
4 · Refresh
Kiểm phiên, cấp access mới
5 · Logout cần bổ sung
Thu hồi access và refresh/phiên
Tham số 15 phút/30 ngày: minh họa
```

REVOCATION — LƯU jti VÀO REDIS

Blocklist thu hồi jti được ghi. Logout còn phải vô hiệu refresh token/phiên liên quan; ví dụ chưa cài route logout.

```python
@jwt.token_in_blocklist_loader
def check_revoked(_h, claims):
    jti = claims["jti"]
    return redis.get(
        f"revoked:{jti}"
    ) is not None
```

JWT trong Flask — protected và refresh

app.py — Flask-JWT-Extended

```python
# Protected — check scope từ claims
@app.get("/api/orders")
@jwt_required()
def list_orders():
    uid = get_jwt_identity()
    claims = get_jwt()
    if "orders:read" not in claims.get("scopes", []):
        return jsonify(error="insufficient_scope"), 403
    return jsonify(orders_by_owner(uid))

# Refresh — chấp nhận refresh, cấp access mới
@app.post("/api/auth/refresh")
@jwt_required(refresh=True)
def refresh():
    # Bổ sung kiểm tra phiên/thu hồi; nạp quyền hiện tại.
    claims = load_permissions(get_jwt_identity())
    return jsonify(access=create_access_token(
        identity=get_jwt_identity(), additional_claims=claims))
```

FLOW NGƯỜI DÙNG

Vòng đời một phiên

```
1 · Login
POST /api/auth/login
→ {access, refresh}
2 · Gọi API
Authorization: Bearer <access>
3 · Access hết hạn
→ cần refresh hoặc đăng nhập lại
4 · Refresh
Kiểm phiên, cấp access mới
5 · Logout cần bổ sung
Thu hồi access và refresh/phiên
Tham số 15 phút/30 ngày: minh họa
```

REVOCATION — LƯU jti VÀO REDIS

Blocklist thu hồi jti được ghi. Logout còn phải vô hiệu refresh token/phiên liên quan; ví dụ chưa cài route logout.

```python
@jwt.token_in_blocklist_loader
def check_revoked(_h, claims):
    jti = claims["jti"]
    return redis.get(
        f"revoked:{jti}"
    ) is not None
```

Bảy rủi ro cần kiểm tra khi dùng JWT

LỖI 01 · CRITICAL

Chấp nhận alg=none

Ứng dụng có thể bị bypass nếu tin alg do token chỉ định mà không giới hạn thuật toán và yêu cầu kiểm chứng.

Đối chiếu alg với allowlist cấu hình tin cậy khi verify.

LỖI 02 · CRITICAL

Algorithm confusion HS↔RS

Server có *public key RSA* cho RS256. Attacker gửi token HS256, ký bằng *chính public key đó* (dùng như HMAC secret) → server verify được bằng public key → **nhận nhầm**.

Phòng tránh: *không* dùng chung một secret cho hai thuật toán.

LỖI 03 · HIGH

Không exp hoặc quá xa

Access token sống lâu làm tăng thời gian bị lợi dụng nếu lộ và chưa bị thu hồi.

```
Ví dụ:
access 15 phút
refresh 30 ngày, có thu hồi
```

LỖI 04 · HIGH

Không rotate secret

Lộ khóa ký cho phép kẻ tấn công tạo token giả được hệ thống chấp nhận cho tới khi khóa bị loại bỏ.

kid là header chỉ định khóa. Chồng lấn khóa tùy TTL; khóa đã lộ cần ngừng tin cậy sớm.

LỖI 05 · HIGH

Lưu token ở localStorage

Mọi JS trong page có thể document.localStorage.getItem('token'). XSS → bay token. Một số team dùng sessionStorage nhưng cũng tương tự.

Với web: cân nhắc BFF và cookie HttpOnly/Secure, SameSite phù hợp; vẫn cần chống CSRF và XSS.

LỖI 06 · MEDIUM

Đặt PII vào payload

Payload của JWT ký đọc được. Chỉ đưa dữ liệu cần thiết, cân nhắc quyền riêng tư và độ cũ của claims.

Không đặt mật khẩu, secret hoặc PII không cần thiết trong claims.

LỖI 07 · MEDIUM

Quên verify aud/iss

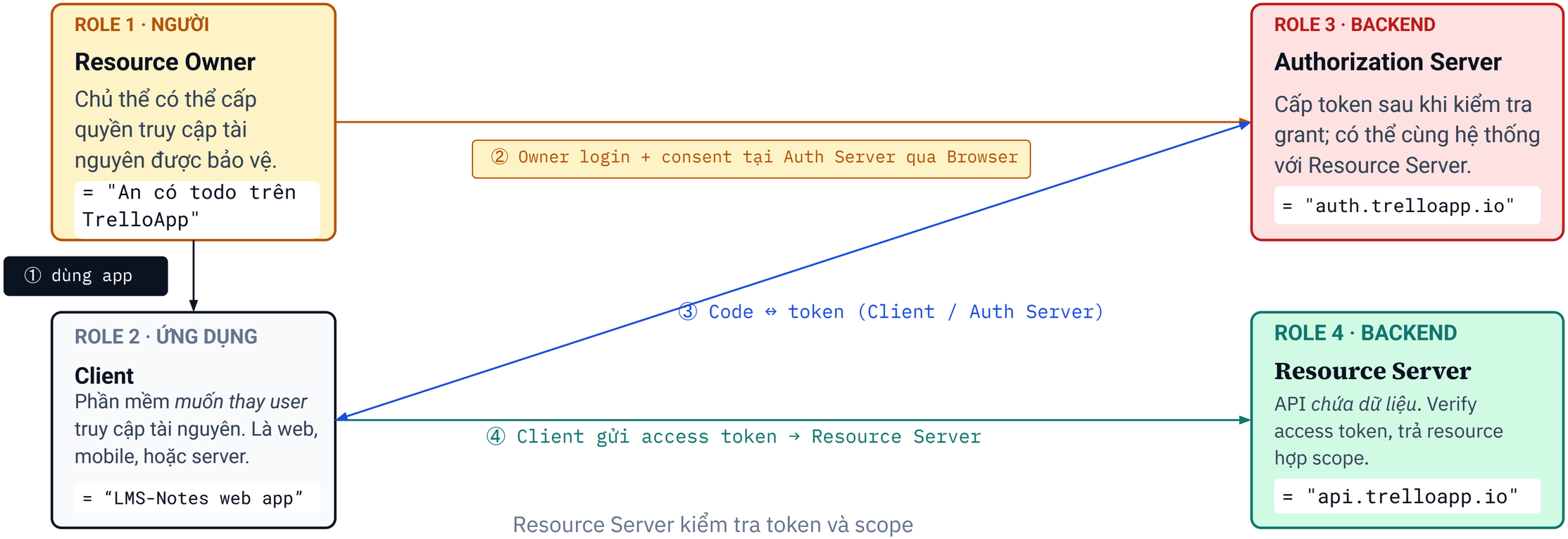
Token có thể bị dùng nhầm dịch vụ nếu bên nhận tin cùng khóa mà không ràng buộc issuer và audience.

Kiểm issuer và audience dự kiến theo profile; iss không nhất thiết là URL.

Chọn thư viện được duy trì · allowlist thuật toán · TTL theo rủi ro · khóa HS256 ngẫu nhiên ≥256 bit · không tự viết crypto.

OAuth 2.0 — bốn vai trò, một mục đích

OAuth 2.0 là framework ủy quyền (RFC 6749). Ví dụ sau dùng Authorization Code để ủy quyền truy cập dữ liệu.



Ví dụ giả định: LMS-Notes nhận quyền đọc todo của An. Phạm vi và thời hạn do chính sách máy chủ quyết định.

Trong flow Authorization Code, người dùng xác thực tại Authorization Server; Client không nhận mật khẩu của họ.

Grant và refresh — chọn theo tình huống

RFC 6749 có bốn grant gốc. Refresh là cơ chế đổi token riêng; PKCE mở rộng Authorization Code.

1
MOST USED

GRANT TYPE

Authorization Code + PKCE

Web, SPA hoặc mobile dùng Authorization Code + PKCE. Public client không giữ client_secret; confidential client xác thực tại token endpoint.

RFC 9700: tránh Implicit; không dùng Password grant. PKCE cũng hữu ích cho confidential client.

2
M2M

GRANT TYPE

Client Credentials

Service-to-service *không có user*. Cron job gọi API nội bộ, gateway nói chuyện với microservice.

```
POST /token
grant_type=client_credentials
client_id=… client_secret=…
→ đổi trực tiếp lấy access token
```

3
PHỤ

REFRESH

Refresh token flow

Đổi refresh → access mới *không cần user re-login*. Cặp (access ngắn, refresh dài) giảm blast radius khi access lộ.

PKCE: code_verifier dùng khi đổi authorization code; không dùng khi refresh.

4
TRÁNH

FLOW CŨ

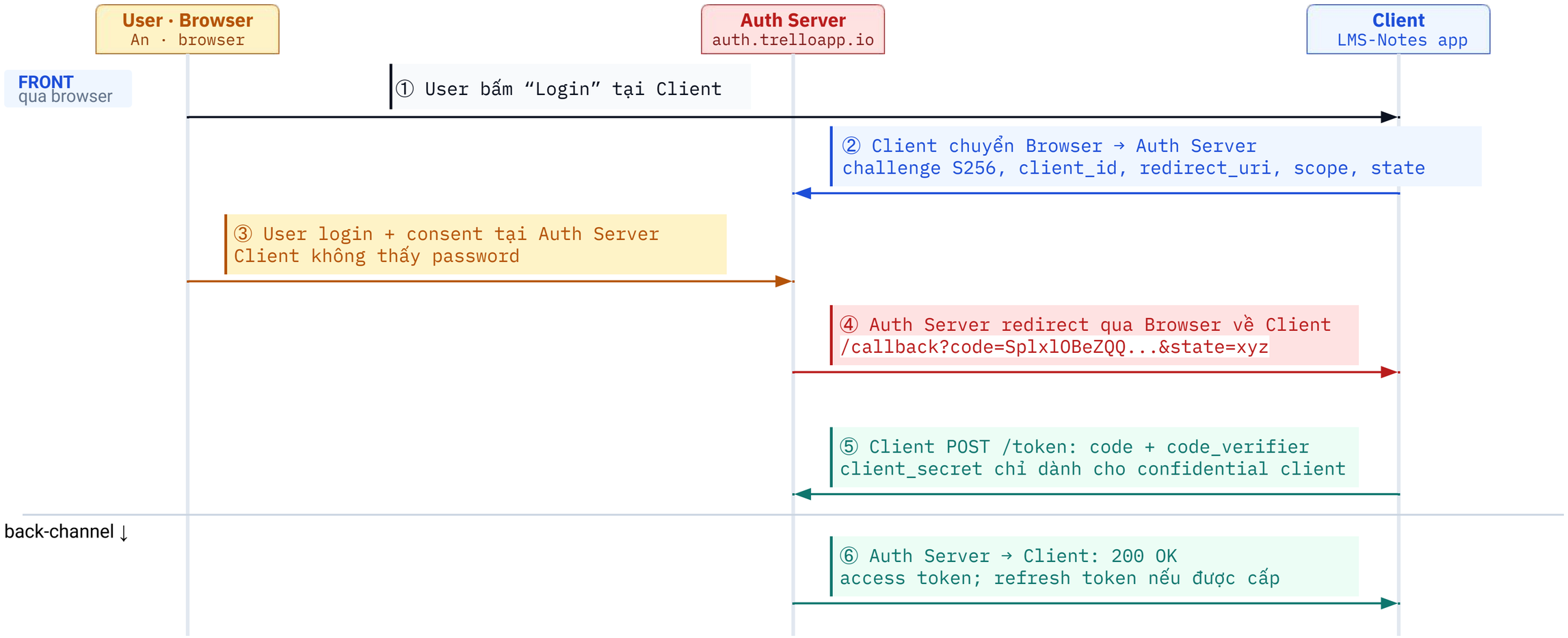
Implicit & Resource Owner Password

- Implicit: SHOULD NOT do rủi ro rò và replay token.
- Password grant: MUST NOT vì đưa credential người dùng cho Client.

Không có user: cân nhắc Client Credentials. Có user: Authorization Code + PKCE; xác thực client theo loại public/confidential.

Authorization Code + PKCE — sáu bước theo dòng

Ví dụ Authorization Code + PKCE S256. Token endpoint dùng HTTPS; public client gọi trực tiếp, confidential client xác thực riêng.



⚠ PKCE CHỐNG CODE CAPTURE

Auth Server kiểm BASE64URL(SHA256(code_verifier)) với code_challenge gốc. Attacker chỉ có code sẽ không đổi được token.

OAuth 2.0 vs JWT — hai thứ làm việc *khác* nhau

OAuth 2.0 là **protocol/framework**. JWT là **token format**. Hai khái niệm orthogonal — thường *dùng chung*, hiếm khi thay thế nhau.

RFC 6749 · RFC 9700 (Security BCP)

OAuth 2.0

"Tôi muốn cho phép."

Một **authorization framework** định nghĩa:

- Cách *client* được user ủy quyền
- Quy trình trao đổi authorization → access token
- Cách *scope* được phát biểu & giới hạn

> **KHÔNG nói** "token trông thế nào". Một OAuth server có thể cấp opaque token, JWT, hoặc PASETO — tùy chọn.

GIAO NHAU THỰC TẾ

Cùng nhau thường được dùng là

OAuth 2.0

cấp token dạng

JWT

OAuth có thể dùng opaque access token hoặc JWT; cần đọc profile của từng dịch vụ.

JWT

"Token của tôi trông thế này."

Một **token format** định nghĩa:

- JWT ký dạng JWS compact có ba phần
- Claims JSON theo profile ứng dụng
- JWS quy định thuật toán bảo vệ

> **KHÔNG nói** "token đến từ đâu, dùng khi nào". JWT có thể vận hành ngoài OAuth hoàn toàn — ví dụ internal API.

BA CÁCH HAY NHẦM — VÀ CÁCH TRÁNH

NHẦM #1

"OAuth là JWT" — sai. OAuth *dùng* JWT là một trong nhiều lựa chọn.

Đề xuất: "API của tôi dùng OAuth 2.0 grant Auth Code, trả về JWT access 15 phút"

NHẦM #2

JWT ký không che nội dung payload.

Giải mã cục bộ; không dán token thật lên website.

NHẦM #3

"OAuth = OpenID" — sai. OpenID Connect *(OIDC)* mới là lớp identity.

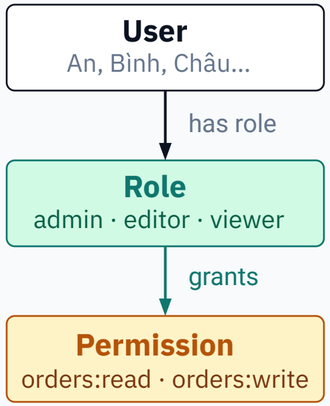
OIDC thêm ID Token; UserInfo dùng khi cần.

Hai mô hình phân quyền — RBAC và ABAC

Đã biết *bạn là ai*. Bây giờ cần quyết định *bạn được làm gì*. Hai trường phái chính: gắn quyền vào **vai trò** (RBAC) hoặc vào **chính sách** (ABAC).

ROLE-BASED ACCESS CONTROL

RBAC — quyền theo vai trò




```
roles ↔ permissions
admin  → orders:rw,
         users:r, billing:r
editor → orders:rw, logs:r
viewer → orders:r
auditor → logs:r, users:r

users ↔ roles
an    → [admin, auditor]
bình → [editor, viewer]
```

DÙNG KHI

Hệ thống có số role *cố định* trong một tổ chức (admin, staff, customer). Quyết định "ai làm gì" **ít thay đổi**. VD: SaaS nội bộ, ERP, CMS doanh nghiệp.

ATTRIBUTE-BASED ACCESS CONTROL

ABAC — quyền theo thuộc tính

SUBJECT

```
role = editor
dept = news
seniority = 3y
```

RESOURCE

```
type = article
owner_id = 5821
draft = false
```

ENVIRONMENT

```
time = 14:32
ip_vpn = true
geofence = VN
```

POLICY
QUYẾT ĐỊNH

```
IF subject.role == "editor"
AND resource.owner_id
    == subject.id
AND env.time >= 09:00
AND env.time <= 18:00
THEN allow: update
```

DÙNG KHI

Quyết định *phụ thuộc nhiều thuộc tính* (thời gian, địa điểm, mối quan hệ với resource, sensitivity). VD: tài liệu y tế HIPAA, hệ thống fintech đa jurisdiction, multi-tenant SaaS.

RBAC phù hợp khi vai trò mô tả tốt quyền. Kết hợp kiểm tra thuộc tính khi cần quyền theo đối tượng, tenant hoặc ngữ cảnh.

RBAC + Scope trong Flask — roles (OR)

Code minh họa từng phần; helper DB, policy và middleware cần được tích hợp riêng.

auth.py — decorators

```python
# Decorator 1: roles (OR) – coarse
from functools import wraps
from flask import g, jsonify
# Middleware phải nạp g.current_user từ DB.

def require_roles(*allowed):
    def d(fn):
        @wraps(fn)
        def w(*a, **kw):
            urs = set(g.current_user["roles"])
            if not urs & set(allowed):
                return jsonify({
                    "error": "forbidden",
                    "need_one_of": list(allowed)
                }), 403
            return fn(*a, **kw)
        return w
    return d
```

3 LỚP CHECK — THEO THỨ TỰ

Stack thành vài lớp

1. **AuthN**
   Token còn hiệu lực, chưa revoke
2. **AuthZ · scope**
   Quyền hành động theo policy
3. **Ownership**
   Quyền truy cập object/tenant

Thiếu kiểm tra quyền đối tượng có thể gây BOLA dù token và scope hợp lệ.

OWASP API1 · BOLA

Broken Object Level Auth

```
GET /api/orders/<id>
Bearer token của user A
Không được đọc order riêng của B
```

```python
# Policy minh họa cho orders
@user_owns_resource("id")
def get_order(id): ...
```

OWASP API1:2023 yêu cầu kiểm quyền đối tượng ở mọi endpoint thao tác object bằng ID.

RBAC + Scope trong Flask — scopes (AND) và route

Code minh họa từng phần; helper DB, policy và middleware cần được tích hợp riêng.

auth.py — decorators

```python
# Decorator 2: scopes (AND) — fine per request
def require_scopes(*needed):
    def d(fn):
        @wraps(fn)
        def w(*a, **kw):
            granted = set(get_jwt().get("scopes", []))
            if not set(needed).issubset(granted):
                return jsonify({
                    "error": "insufficient_scope",
                    "need_all_of": list(needed)
                }), 403
            return fn(*a, **kw)
        return w
    return d

# 3 lớp check chồng lên một route
@app.delete("/api/orders/<int:id>")
@jwt_required()
@require_roles("admin", "manager")
@require_scopes("orders:write")
@user_owns_resource("id")
def delete_order(id):
    order_svc.delete_authorized(id, g.current_user)
    return "", 204
```

3 LỚP CHECK — THEO THỨ TỰ

Stack thành vài lớp

1. **AuthN**
   Token còn hiệu lực, chưa revoke
2. **AuthZ · scope**
   Quyền hành động theo policy
3. **Ownership**
   Quyền truy cập object/tenant

Thiếu kiểm tra quyền đối tượng có thể gây BOLA dù token và scope hợp lệ.

OWASP API1 · BOLA

Broken Object Level Auth

```
GET /api/orders/<id>
Bearer token của user A
Không được đọc order riêng của B
```

```python
# Policy minh họa cho orders
@user_owns_resource("id")
def get_order(id): ...
```

OWASP API1:2023 yêu cầu kiểm quyền đối tượng ở mọi endpoint thao tác object bằng ID.

RBAC + Scope trong Flask — ownership chống BOLA

Code minh họa từng phần; helper DB, policy và middleware cần được tích hợp riêng.

auth.py — decorators

```python
# Decorator 3: ownership — chống BOLA (API1)
def user_owns_resource(param="id"):
    def d(fn):
        @wraps(fn)
        def w(*a, **kw):
            row = db.fetchone(
                "SELECT owner_id FROM orders WHERE id=?",
                (kw[param],))
            if not row:
                return jsonify({"e": "nf"}), 404
            ok = str(row["owner_id"]) == str(g.current_user["id"])
            if not ok and "admin" not in g.current_user["roles"]:
                return jsonify({"error": "forbidden"}), 403
            return fn(*a, **kw)
        return w
    return d
```

3 LỚP CHECK — THEO THỨ TỰ

Stack thành vài lớp

1. **AuthN**
   Token còn hiệu lực, chưa revoke
2. **AuthZ · scope**
   Quyền hành động theo policy
3. **Ownership**
   Quyền truy cập object/tenant

Thiếu kiểm tra quyền đối tượng có thể gây BOLA dù token và scope hợp lệ.

OWASP API1 · BOLA

Broken Object Level Auth

```
GET /api/orders/<id>
Bearer token của user A
Không được đọc order riêng của B
```

```python
# Policy minh họa cho orders
@user_owns_resource("id")
def get_order(id): ...
```

OWASP API1:2023 yêu cầu kiểm quyền đối tượng ở mọi endpoint thao tác object bằng ID.

Hash password như *thế nào* — và *đừng* như thế nào

Hash mật khẩu làm tăng chi phí đoán mật khẩu khi dữ liệu lưu trữ bị lộ. Salt riêng ngăn tái sử dụng kết quả tính sẵn.

KHÔNG DÙNG TRỰC TIẾP

MD5 · SHA-1 · SHA-256 — quá nhanh

```
Hash("password") = 5f4dcc3b5aa765d61d8327deb882cf99 ← mọi máy cùng ra một kết quả, không salt
```

Hàm hash tổng quát được thiết kế chạy nhanh

TRÁNH TỰ THIẾT KẾ

Dùng hàm dẫn xuất khóa đã được phân tích

Salt không cần bí mật. Lặp hash tự chế không thay thế PBKDF2 hoặc hàm password hashing chuẩn.

Không tự lặp SHA-256 để hash password

HỆ THỐNG ĐANG DÙNG BCRYPT

bcrypt — duy trì đúng cấu hình

```
Hash("password") = $2b$12$N9qo8uLOickgx2Z… ← salt + cost 12
```

OWASP: work factor ≥10 cho bcrypt legacy; đo trên hạ tầng để chọn cost phù hợp.

Cost điều chỉnh được; mật khẩu giới hạn 72 byte

ƯU TIÊN CHO HỆ THỐNG MỚI

Argon2id — biến thể của Argon2

```
$argon2id$v=19$m=65536,t=3,p=4$saltsalt…$hashhash…
```

Memory-hard, tăng chi phí đoán song song

CHẤP NHẬN ĐƯỢC nếu bắt buộc FIPS

PBKDF2-HMAC · pbkdf2_sha256 ≥ 600.000 iterations

Cần triển khai/module đáp ứng yêu cầu FIPS

Đo hiệu năng dưới tải; nâng work factor và rehash khi đăng nhập nếu cần.

Hash password — ví dụ Flask với Argon2id

Dùng thư viện tạo salt, hash và xác minh mật khẩu. Thời gian thực thi phụ thuộc máy và tải đồng thời.

```
# pip install argon2-cffi
from argon2 import PasswordHasher, Type
from argon2.exceptions import VerifyMismatchError

ph = PasswordHasher(time_cost=3, memory_cost=65536,
                    parallelism=4, type=Type.ID)
stored = ph.hash("example-password")
try:
    ph.verify(stored, "example-password")
except VerifyMismatchError:
    pass  # trả lỗi đăng nhập chung
else:
    if ph.check_needs_rehash(stored):
        stored = ph.hash("example-password")
```

Không ép đổi mật khẩu định kỳ chỉ vì hash cũ; rehash khi cần nâng tham số.

Bốn nhóm lỗ hổng liên quan đến auth

Ánh xạ OWASP API Security Top 10 2023; ví dụ dưới đây là tình huống minh họa.

OWASP API1:2023
Broken Object Level Authorization (BOLA)

User A đọc tài liệu của B vì API không kiểm quyền đối với tài liệu được yêu cầu. BOLA là mục API1:2023.

> **Chặn:** kiểm quyền trên đối tượng theo chính sách, gồm owner, chia sẻ, tenant và quyền quản trị khi phù hợp.

OWASP API2:2023
Broken Authentication

Ví dụ: kiểm token sai, thiếu bảo vệ login hoặc xử lý credential không an toàn.

> **Chặn:** 1 middleware tập trung; mọi endpoint **fail-closed**; kiểm thử fuzz cho 401; rate-limit cả credential stuffing.

OWASP API5:2023
Broken Function Level Authorization

Endpoint admin chỉ kiểm tra "đã đăng nhập" mà không kiểm tra role. Một user thường gọi POST /admin/users vẫn thành công.

> **Chặn:** route admin phải có decorator @require_roles("admin"); *mặc định deny*, không allow-by-default.

OWASP API3:2023
Broken Object Property Level Auth (mass-assign)

PATCH /users/me với body {email, is_admin:true} → user tự set field *is_admin*. Hoặc để lộ password_hash qua response.

> **Chặn:** whitelist field read/write; không echo raw DB row; serialize qua DTO.

5 ĐIỀU PHẢI CÓ TRƯỚC KHI SHIP

① Phân loại endpoint public/protected; mặc định từ chối

② Lỗi login chung, giảm rò thông tin qua thời gian

③ Rate-limit theo rủi ro; tránh lockout gây từ chối dịch vụ

④ Log sự kiện cần thiết; không log password/token

⑤ Kiểm thử quyền và rà soát khi có thay đổi

Năm điều cần nhớ *về* Auth

Dùng năm ý sau để giải thích các quyết định auth và rà soát ví dụ API trong bài tập.

01 AuthN ≠ AuthZ

Tách rõ trách nhiệm xác thực và quyết định quyền

Có thể chung middleware, nhưng cần kiểm thử riêng từng trách nhiệm.

02 CHỌN THEO CONTEXT

Cookie-session ≠ JWT ≠ OAuth 2.0

Cơ chế phiên, định dạng token và ủy quyền thuộc các lớp khác nhau, có thể kết hợp.

03 JWT CẦN DISCIPLINE

Whitelist alg, set *exp* ngắn, refresh rotate

Chọn thư viện được duy trì; kiểm token theo profile; có kế hoạch thu hồi khóa và token.

04 AUTHZ THEO CHÍNH SÁCH

Danh tính · quyền hành động · quyền đối tượng

Ownership là một chính sách minh họa; dữ liệu chia sẻ và multi-tenant cần quy tắc riêng.

05 HASH MẬT KHẨU ĐÚNG

Argon2id hoặc phương án phù hợp, có salt và cost

Rehash khi cần nâng tham số. Không áp lịch đổi mật khẩu định kỳ thiếu căn cứ.

BUỔI 7 — TIẾP THEO

Nguyên lý triển khai Backend Service

OpenAPI definition × runtime stack × database → working backend. Đem code bài buổi này để build auth service tái sử dụng.

BÀI TẬP VỀ NHÀ — HẠN CUỐI: trước Buổi 7

Cho một API social-network bạn đang xây: viết OpenAPI spec cho 3 endpoint sau — POST /auth/login, POST /auth/refresh, GET /posts/{id}. Câu hỏi phải trả lời trong spec: *mỗi endpoint cần scope nào, lỗi nào có thể xảy ra, status code nào được trả?* Bài nộp kèm commit: **SecurityAudit.md** liệt kê ≥ 5 điểm có thể lộ.